Python Chapter 6 Lists Activities

Contents

Python Chapter 6 Lists Activities 1
DRY 1
Online Tutorials 1
Tutorial 1: Traversing a List with a Loop 2
Tutorial 2: List and String Slicing 4
Tutorial 3: List Functions 6
in and not in 6
List Methods 7
append() 8
insert() 8
sort() 8
reverse() 9
index() 9
remove() 10
pop() 10
extend() 11
Try Except 12
Tutorial 4: Try Except Fahrenheit Converter 13
Assignment 1: List Slicing and Iteration 14
Assignment Submission 14

Time required: 90 minutes

DRY

Don’t Repeat Yourself

Online Tutorials

Go through the following tutorials.

- LearnPython.org Lists
- Python Lists
    - Access List Items
    - Change List Items
    - Add List Items
    - Remove List Items
    - Loop Lists
    - List Comprehensions
    - Sort Lists
    - Copy Lists
    - Join Lists
    - List Methods

Tutorial 1: Traversing a List with a Loop

A List is a sequence of elements. A String is a sequence of elements. A Tuple is a sequence of elements. The major difference is that a List is mutable, it can be changed. Otherwise, each of these sequences behave in similar ways.

The most common way to iterate or traverse the elements of a list is with a for loop. The syntax is the same as for strings.

```
for cheese in cheeses:
    print(cheese)
cheddar
colby
swiss
```

If you want to write or update the elements, you need the indices. A common way to do that is to combine the functions **range** and **len**:

```
# range_len.py
numbers = [1, 3, 5]
for i in range(len(numbers)):
    # Update element in list
    numbers[i] = numbers[i] * 2
    print(f"{i + 1} {numbers[i]}")
```

This loop traverses the list and updates each element. **len** returns the number of elements in the list. **range** returns a list of indices from 0 to n − 1, where **n** is the length of the list. Each time through the loop, **i** gets the index of the next element. The assignment statement in the body uses **i** to read the old value of the element and to assign the new value.

Create a Python program named **list_loops.py**

```
1   """
2       Name: list_loops.py
3       Author:
4       Created:
5       Purpose:
6   """
7   # Define a list of numbers: 1, 3, 5
8   numbers = [1, 3, 5]
9
10  # Iterate through each number in the list and print it
11  for number in numbers:
12      print(f"{number}")
13
14  # Iterate through the indices of the list using a for loop
15  for i in range(len(numbers)):
16      # Multiply each number in the list by 2
17      numbers[i] = numbers[i] * 2
18
19      # Print the modified numbers along with their respective indices
20      print(f"{i + 1}. {numbers[i]}")
```

Example run:

```
1
3
5
1. 2
2. 6
3. 10
```

Tutorial 2: List and String Slicing

The Python slice operator, represented by the colon **:**, allows you to extract a portion of a sequence like a string, list, or tuple by specifying a start, stop, and step size within square brackets [ ].

The general syntax for slicing is: **sequence[start:stop:step]**

```
new_list = original_list[start:stop:step]
new_string = original_string[start:stop:step]
```

- **start (optional)**: The index where the slice begins (inclusive). If omitted, it starts from the beginning of the sequence (index 0).
- **stop (optional)**: The index where the slice ends (exclusive). If omitted, it goes until the end of the sequence.
- **step (optional)**: The increment between indices while slicing. If omitted, it defaults to 1.

```
t = ["a", "b", "c", "d", "e", "f"]
print(t[1:3])
# Output: ['b', 'c']

print(t[:4])
# Output: ['a', 'b', 'c', 'd']

print(t[3:])
# Output: ['d', 'e', 'f']
```

- `t[1:3]` slices the index 1 and 2 items.
- `t[:4]` slices everything from the beginning of the list to index 3.
- `t[3:]` slices everything after and including index 3.

If you omit the first index, the slice starts at the beginning. If you omit the second, the slice goes to the end. If you omit both, the slice is a copy of the whole list.

```
print(t[:])
# Output: ['a', 'b', 'c', 'd', 'e', 'f']
```

Lists and strings can be sliced in simlar ways. Create a Python program named **list_slice.py**

```python
"""
    Name: list_slice.py
    Author:
    Created:
    Purpose: Demonstrate Python slicing
"""

# Example 1: Slicing a range from a list
numbers = [1, 2, 3, 4, 5]
subset = numbers[1:4]
print(subset)

# Example 1: Slicing a range from a string
text = "PythonProgramming"
subset = text[0:6]
print(subset)
```

Example run:

```
[2, 3, 4]
Python
```

```python
# Example 2: Slicing with step
even_numbers = numbers[::2]
print(even_numbers)

# Example 2: Slicing with step
every_second_char = text[::2]
print(every_second_char)  # Output: PtoPormig

# Example 3: Slicing from the end
last_three = numbers[-3:]
print(last_three)

# Example 4: Modifying elements
fruits = ["apple", "banana", "cherry", "date"]
fruits[1:3] = ["pear", "grape"]
print(fruits)
```

Example run:

```
[1, 3, 5]
PtoPormig
[3, 4, 5]
['apple', 'pear', 'grape', 'date']
```

Tutorial 3: List Functions

There are many built-in functions that operate on lists without writing your own loops.

Function	Description
len()	Returns the number of items in the list
sum()	Returns the sum of the items in the list
min()	Returns the minimum of the items in the list
max()	Returns the maximum of the items in the list
sorted()	Returns a sorted list

in and not in

The in operator tells you if a list or string contains something. The not in operator tells you if the list of string doesn't contain something.

Create a Python file named **list_functions.py**

```python
nums = [3, 41, 12, 9, 74, 15]

print(len(nums))

print(max(nums))

print(min(nums))

print(sum(nums))

print(sum(nums) / len(nums))

print(sorted(nums))

# The in operator tells you if your list or string contains an element
if 3 in nums:
    print("Your list contains the number 3. ")

# not in tells you if your list or string does not contain an element
if 0 not in nums:
    print("Your list has no zeroes.")
```

Example run:

```
6
74
3
154
25.666666666666668
[3, 9, 12, 15, 41, 74]
Your list contains the number 3.
Your list has no zeroes.
```

List Methods

Python provides methods that operate on lists. Methods use the **.** (dot) operator.

Method	Description
append(x)	add **x** to the end of the list
sort(x)	sort the list in place
count(x)	return the number of times **x** occurs in the list
index(x)	return the location of the first occurrence of **x**

reverse()	reverse the list in place
clear()	remove all the elements from the list
remove(x)	remove the first occurrence of **x** from the list
pop(p)	remove the item at index **p** and return its value
insert(p, x)	insert **x** at index **p** of the list

Create a Python file named **list_methods.py** Add each of these code snippets to the program.

append()

Append adds a new element to the end of a list:

```
my_list = ["a", "b", "c"]
print(my_list)
my_list.append("d")
print(f"Append: {my_list}")
```

Example run:

```
Original: ['a', 'b', 'c']
Append: ['a', 'b', 'c', 'd']
```

insert()

Insert places a new element at the designate index and moves the others aside.

```
my_list.insert(1, "d")
print(f"Insert: {my_list}")
```

Example run:

```
Original: ['a', 'b', 'c']
Append: ['a', 'b', 'c', 'd']
Insert: ['a', 'd', 'b', 'c', 'd']
```

sort()

Arranges the elements of the list from low to high:

```
# Sorts list in place
my_list.sort()
print(f"Sort: {my_list}")
```

Example run:

```
Original: ['a', 'b', 'c']
Append: ['a', 'b', 'c', 'd']
Insert: ['a', 'd', 'b', 'c', 'd']
Sort: ['a', 'b', 'c', 'd', 'd']
```

reverse()

The reverse() reverses the original order of the list. It doesn't sort backward alphabetically, it reverses the current order of the list,

```
my_list.reverse()
print(f"Reverse: {my_list}")
```

Example run:

```
Original: ['a', 'b', 'c']
Append: ['a', 'b', 'c', 'd']
Insert: ['a', 'd', 'b', 'c', 'd']
Sort: ['a', 'b', 'c', 'd', 'd']
Reverse: ['d', 'd', 'c', 'b', 'a']
```

index()

Returns the index of the first occurrence on the list of the element that is given to index() as argument. A runtime error is generated if the element is not found on the list.

```
print(f"Index: {my_list.index("b")}")
```

Example run:

```
Original: ['a', 'b', 'c']
Append: ['a', 'b', 'c', 'd']
Insert: ['a', 'd', 'b', 'c', 'd']
Sort: ['a', 'b', 'c', 'd', 'd']
Reverse: ['d', 'd', 'c', 'b', 'a']
Index: 3
```

count()

Returns an integer that indicates how often the element that is passed to it as an argument occurs in the list.

```
print(f"Count: {my_list.count("b")}")
```

Example run:

```
Original: ['a', 'b', 'c']
Append: ['a', 'b', 'c', 'd']
Insert: ['a', 'd', 'b', 'c', 'd']
Sort: ['a', 'b', 'c', 'd', 'd']
Reverse: ['d', 'd', 'c', 'b', 'a']
Index: 3
Count: 1
```

remove()

Removes an item with a certain value.

```
my_list.remove("b")
print(f"Remove b: {my_list}")
```

Example run:

```
Original: ['a', 'b', 'c']
Append: ['a', 'b', 'c', 'd']
Insert: ['a', 'd', 'b', 'c', 'd']
Sort: ['a', 'b', 'c', 'd', 'd']
Reverse: ['d', 'd', 'c', 'b', 'a']
Index b: 3
Count b: 1
Remove b: ['d', 'd', 'c', 'a']
```

pop()

pop removes an item from a list at a designated index and returns a value. You can assign that value to a variable. If you don't provide an index, it deletes and returns the last element.

```
popped = my_list.pop(1)
print(f"Pop {popped}: {my_list}")
```

Example run:

```
Original: ['a', 'b', 'c']
Append: ['a', 'b', 'c', 'd']
Insert: ['a', 'd', 'b', 'c', 'd']
Sort: ['a', 'b', 'c', 'd', 'd']
Reverse: ['d', 'd', 'c', 'b', 'a']
Index b: 3
Count b: 1
Remove b: ['d', 'd', 'c', 'a']
Pop d: ['d', 'c', 'a']
```

extend()

Takes a list as an argument and appends all the elements. This example leaves t2 unmodified.

```
my_list.extend(my_list)
print(f"Extend: {my_list}")
```

Example run:

```
Original: ['a', 'b', 'c']
Append: ['a', 'b', 'c', 'd']
Insert: ['a', 'd', 'b', 'c', 'd']
Sort: ['a', 'b', 'c', 'd', 'd']
Reverse: ['d', 'd', 'c', 'b', 'a']
Index b: 3
Count b: 1
Remove b: ['d', 'd', 'c', 'a']
Pop d: ['d', 'c', 'a']
Extend: ['d', 'c', 'a', 'd', 'c', 'a']
```

join()

.join() is not a list method, it is a string method. It does work very well with lists. **.join()** iterates through the list joining each string with the separator.

```
separator = " | "
my_list = separator.join(my_list)
print(f"Separator: {my_list}")
```

Example run:

```
Original: ['a', 'b', 'c']
Append: ['a', 'b', 'c', 'd']
Insert: ['a', 'd', 'b', 'c', 'd']
Sort: ['a', 'b', 'c', 'd', 'd']
Reverse: ['d', 'd', 'c', 'b', 'a']
Index b: 3
Count b: 1
Remove b: ['d', 'd', 'c', 'a']
Pop d: ['d', 'c', 'a']
Extend: ['d', 'c', 'a', 'd', 'c', 'a']
Separator: d | c | a | d | c | a
```

Try Except

Things go wrong, errors happen, let's learn to handle those exceptional events with try except.

An exception is a signal that a condition has occurred that can't be easily handled using the normal flow-of-control of a Python program. Exceptions are often defined as being "errors" but this is not always the case. All errors in Python are dealt with using exceptions, but not all exceptions are errors.

Go through this short tutorial for a better understanding of exceptions.

- Python Try Except

Basic syntax of exception handling

```
try:
    # Code that might raise an exception
except Exception as e:
    # Code to handle the exception
```

Example of

Here is an example of a program without exception handling.

```
imput = input('Enter Fahrenheit Temperature: ')
fahrenheit = float(input)
celsius = ((fahrenheit - 32.0) * 5.0) / 9.0
print(celsius)
```

If the user types in a float, the program works just fine. If the user types in anything else, a **ValueError** exception occurs.

Example run:

```
Enter Fahrenheit Temperature: d
Traceback (most recent call last):
  File "Z:\_WNCC\Computer Science\Assignments Python\0
aries\Tutorials\temperature_converter.py", line 2, in
    fahrenheit = float(input)
ValueError: could not convert string to float: 'd'
```

Let's rewrite this program to catch and handle exceptions.

Tutorial 4: Try Except Fahrenheit Converter

Create and test the following program with correct and incorrect input.

```python
"""
    Name: temperature_converter.py
    Author:
    Created:
    Purpose: Handle ValueError and general exceptions
"""

try:
    # If the user doesn't type a number,
    # an exception is raised
    fahrenheit = float(input("Enter Fahrenheit Temperature: "))

    # These lines execute if an exception isn't raised
    celsius = ((fahrenheit - 32.0) * 5.0) / 9.0
    print(celsius)

except ValueError as v:
    # This is a specific exception
    # You don't have to have a specific exception handler
    # Handle ValueError, print out exception
    print(f"Please enter a whole number. \n{v}")

except Exception as e:
    # Handle any other exception, print out exception message
    # You can just use this general exception handling
    print(f"Something went wrong. \n{e}")
```

Exceptions are now handled.

Example run:

```
Enter Fahrenheit Temperature: b
Please enter a whole number.
could not convert string to float: 'b'
```

Assignment 1: List Slicing and Iteration

Objective: Understand list slicing and iteration.

1. Create a list named numbers with the numbers from 1 to 10.
2. Print the first five numbers using list slicing.
3. Print the last three numbers using list slicing.
4. Use a loop to print each number in the list.
5. Use a loop to print each number in the list that is greater than 5.

Example run:

```
[1, 2, 3, 4, 5]
[8, 9, 10]
1
2
3
4
5
6
7
8
9
10
6
7
8
9
10
```

Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the programs.
3. Submit in Blackboard.